

Modeling Song Popularity with Neural Networks and Bayesian Regression

DS4420

Prepared by: Siyuan Chen, Yifan Wang

Instructor: Dr. Eric Gerber

Date: April 16, 2026

Introduction

Music popularity is an important topic in the music industry. Platforms like Spotify rely on popularity to recommend songs. Artists also care about popularity because it reflects audience interest. However, it is not always clear what makes a song popular.

This project studies whether audio features can help predict song popularity. We focus on five features: danceability, energy, tempo, loudness, and valence. These features describe how a song sounds and feels. The goal is to test if these characteristics are related to popularity.

To answer this question, we use two machine learning approaches. First, we build a neural network model. This model can capture non-linear patterns in the data. Second, we use a Bayesian regression model. This model helps explain how each feature affects popularity. It also provides uncertainty for the estimates. By comparing these two methods, we aim to understand both prediction and interpretation. We want to see how well audio features can predict popularity. We also want to identify which features matter the most. This comparison gives a more complete view of the problem.

The full implementation and additional materials for this project are available on GitHub:

<https://github.com/Miles1228/music-DS4420-project.git>

Literature Review

Many studies have explored how music features relate to song popularity. Early work often focused on statistical models. More recent studies use machine learning methods, which can capture more complex patterns in the data.

One line of research uses audio features to predict music success. Herremans et al. (2014) applied classification models to dance hit prediction using musical features. Their results show that audio characteristics can help distinguish top-ranked songs from lower-ranked ones. However, the relationships between audio features and popularity are not always strong. External factors also play a role, and audio features alone cannot fully explain the outcome.

Other research applies neural networks to music-related tasks. Neural networks can capture complex patterns that simpler models cannot detect. However, these models are often difficult to interpret. It is not easy to explain why a particular feature leads to a certain prediction.

Bayesian methods provide a different approach. Instead of focusing only on prediction, they emphasize interpretation. Bayesian regression produces coefficient estimates and credible intervals, which show the uncertainty of each estimate (Gelman et al., 2013). This makes Bayesian models useful for understanding the relationships between features and outcomes.

Despite these advances, most studies agree on one limitation. Music popularity is not determined only by audio features. Factors such as artist reputation, marketing, and social media exposure play a major role. Models based only on audio features usually have limited predictive power.

In this project, we combine these two approaches. The neural network focuses on prediction. The Bayesian model focuses on interpretation. By comparing these methods, we aim to

better understand both the strengths and the limitations of using audio features to model song popularity.

ML Methodology and Data Collection

Data Section

This project uses the Spotify Tracks Dataset. The dataset contains about 114,000 songs. Each song includes several audio features and a popularity score.

The target variable is popularity. It ranges from 0 to 100. This value reflects listener engagement and streaming activity. We select five audio features as predictors. These include danceability, energy, tempo, loudness, and valence. These variables describe different aspects of a song's sound.

The dataset is cleaned before modeling. Rows with missing values are removed. No data loss occurs because the selected features contain no null values. The data is then split into training and test sets. We use an 80/20 split.

All features are standardized using the training set mean and standard deviation. The same transformation is applied to the test set. This step ensures stable model training and prevents data leakage.

Neural Network Method

We implement a neural network model using Python and NumPy. The model is built from scratch without using pre-built machine learning libraries.

The network has three layers. The input layer contains 5 nodes. Two hidden layers contain 64 and 32 nodes. The output layer has 1 node for regression.

ReLU is used as the activation function in the hidden layers. The output layer uses a linear function. This allows continuous predictions. The model is trained using mini-batch stochastic gradient descent. The batch size is 256. The learning rate is 0.001. The training data is shuffled at each epoch.

Weights are initialized using He initialization. This helps stabilize gradients during training. Forward propagation computes predictions. Backpropagation computes gradients using the chain rule. The model is trained for 500 epochs.

Bayesian Regression Method

We apply a Bayesian linear regression model in R using `stan_glm`. The model assumes a linear relationship between the five audio features and popularity. Errors are assumed to follow a normal distribution. Each coefficient is given a weakly informative normal prior centered at zero. This allows the posterior to be driven mainly by the data.

Posterior distributions are estimated using Hamiltonian Monte Carlo sampling. We run 4 chains with 2,000 iterations each. Convergence is checked using the Gelman-Rubin statistic. All parameters reach $\hat{R} = 1.0$, which indicates the chains have converged. All features are standardized before fitting, so the coefficients can be compared directly across variables.

Model Comparison Approach

The two models are designed for different purposes. The neural network focuses on prediction. The Bayesian model focuses on interpretation.

We compare the models based on two aspects. First, we evaluate predictive performance using metrics such as RMSE and R^2 . Second, we analyze how each model explains the relationship between features and popularity.

ML Results and Interpretation

Neural Network Results

The neural network model was evaluated on the test set using standard regression metrics. The results show an RMSE of 21.52, an MAE of 17.59, and an R^2 of 0.071. These values indicate limited predictive performance.

Figure 1 shows the relationship between predicted and actual popularity. Most predictions are concentrated between 15 and 55, while the actual values range from 0 to 100. The model fails to capture very low and very high popularity scores. This pattern suggests regression toward the mean.

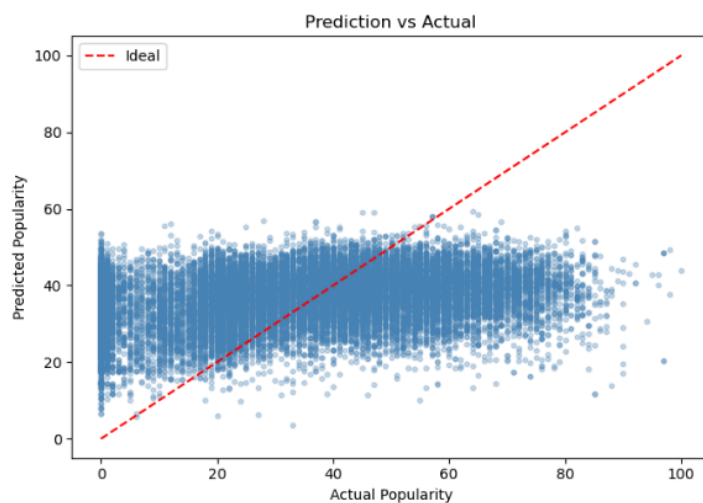
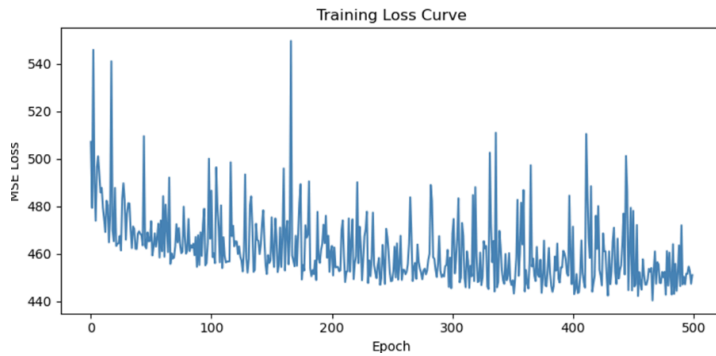


Figure 2 presents the training loss curve. The loss decreases over time, which indicates that the model is learning from the data. However, the improvement slows after several epochs. This suggests that the model reaches a limit in its ability to fit the data.



Bayesian Regression Results

The Bayesian regression model provides estimates for the effect of each feature on popularity. The results show that loudness and danceability have positive effects. Energy and valence have negative effects. Tempo has a small positive effect.

Figure 3 shows the posterior coefficient intervals. All intervals do not include zero. This suggests that each feature has a statistically meaningful relationship with popularity.

The model also provides uncertainty estimates. The credible intervals are relatively narrow. This indicates that the coefficient estimates are stable.

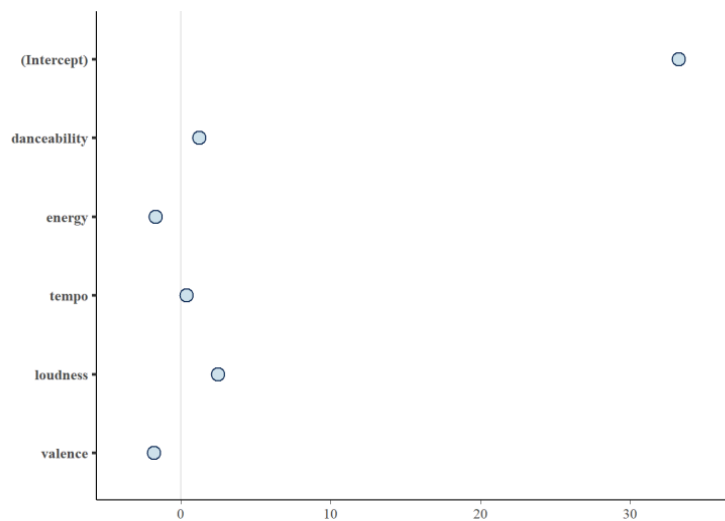
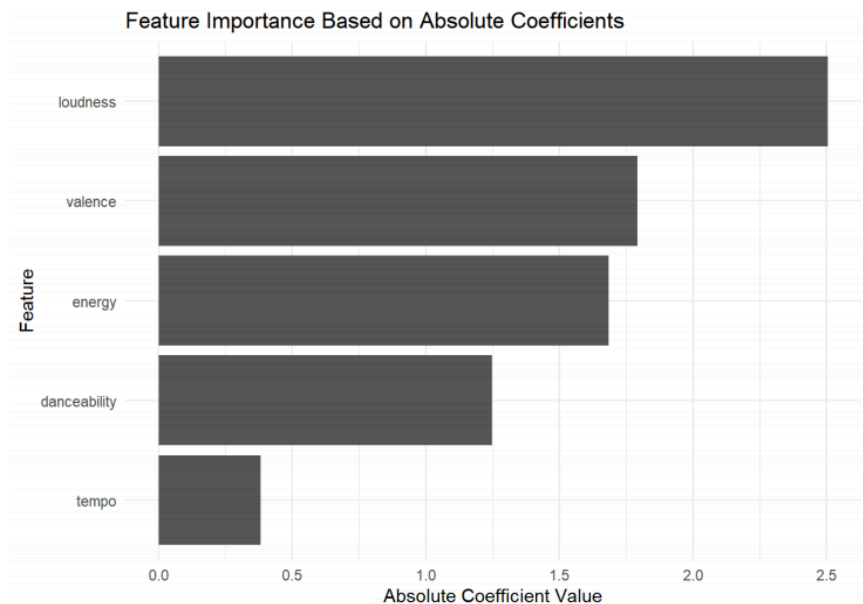


Figure 4 shows the relative importance of each feature based on the estimated coefficients.



Interpretation and Comparison

Both models suggest that audio features have limited predictive power. The neural network produces predictions close to the mean. The Bayesian model shows that the effects of features are consistent but relatively small.

These findings are aligned. The neural network captures general patterns but cannot achieve high accuracy. The Bayesian model explains the direction of feature effects but also indicates that the magnitude of these effects is not large.

One reason for this limitation is that song popularity depends on many external factors. These include artist reputation and playlist placement. None of these variables are included in the dataset.

Overall, the results show that audio features alone are not sufficient to predict song popularity. However, they still provide useful information about general trends in the data.

Conclusions and Future Work

This project examined whether audio features can be used to predict song popularity. Two models were applied: a neural network and a Bayesian regression model. The results show that both models capture some patterns in the data, but their overall predictive performance is limited.

The neural network demonstrates the ability to learn non-linear relationships. However, its predictions are concentrated around the mean, and the R^2 value remains low. This indicates that the model cannot explain much of the variation in popularity. The Bayesian model provides clear interpretation of feature effects. Loudness and danceability show positive relationships, while energy and valence show negative effects. The credible intervals confirm that these relationships are statistically meaningful.

The assumptions of both methods are partially met. The Bayesian model assumes a linear relationship between features and the target. This assumption is likely too simple for modeling popularity, which is influenced by many complex factors. The neural network does not require linearity, but it still relies on the assumption that the selected features contain enough information. In this case, that assumption is not fully satisfied.

There are several drawbacks in this analysis. First, the model only uses five audio features. Important factors such as artist popularity, release timing, and social media influence are not included. Second, the dataset is imbalanced, with many songs having low popularity. This affects model performance and leads to predictions close to the mean. Third, the neural network model may not be fully optimized, and different architectures or hyperparameters could improve results.

Future work could extend this project in several ways. Additional features could be included to better represent real-world factors. More advanced models could be tested, such as deeper neural networks or hybrid approaches. It may also be useful to reformulate the problem as a classification task, for example predicting whether a song will be highly popular. These improvements could lead to better predictive performance and deeper insights.

References

Herremans, D., Martens, D., & Sörensen, K. (2014). Dance hit song prediction. *Journal of New Music Research*.

Gelman, A., et al. (2013). *Bayesian Data Analysis* (3rd ed.). CRC Press.

Appendix

Neural Network Implementation

1. Code

```
class ManualMLP:
    """
    Multi-Layer Perceptron for regression, implemented from scratch using NumPy.
    Architecture : input -> hidden layers (ReLU) -> output (linear)
    Optimizer    : mini-batch stochastic gradient descent
    """

    def __init__(self, layer_sizes, learning_rate=0.001,
                  max_iter=500, batch_size=256, random_state=42):
        np.random.seed(random_state)
        self.layer_sizes = layer_sizes
        self.lr = learning_rate
        self.max_iter = max_iter
        self.batch_size = batch_size
        self.loss_history = []

        # He initialization (suited for ReLU activations)
        self.weights = []
        self.biases = []
        for i in range(len(layer_sizes) - 1):
            w = np.random.randn(layer_sizes[i], layer_sizes[i+1]) * np.sqrt(2.0 / layer_sizes[i])
            b = np.zeros((1, layer_sizes[i+1]))
            self.weights.append(w)
            self.biases.append(b)

        # Activation functions
        def _relu(self, z):
            return np.maximum(0, z)

        def _relu_grad(self, z):
            return (z > 0).astype(float)

        # Forward pass
        def _forward(self, X):
            self._activations = [X]
            self._z_vals = []
            for i, (w, b) in enumerate(zip(self.weights, self.biases)):
                z = self._activations[-1] @ w + b
                self._z_vals.append(z)

                a = self._relu(z) if i < len(self.weights) - 1 else z
                self._activations.append(a)
            return self._activations[-1]
```



```

# Backward pass (backpropagation)
def _backward(self, y_true):
    m = y_true.shape[0]
    y_true = y_true.reshape(-1, 1)

    delta = 2 * (self._activations[-1] - y_true) / m

    grad_w = [None] * len(self.weights)
    grad_b = [None] * len(self.weights)

    for i in reversed(range(len(self.weights))):
        grad_w[i] = self._activations[i].T @ delta
        grad_b[i] = delta.sum(axis=0, keepdims=True)
        if i > 0:
            delta = (delta @ self.weights[i].T) * self._relu_grad(self._z_vals[i-1])

    # Gradient descent weight update
    for i in range(len(self.weights)):
        self.weights[i] -= self.lr * grad_w[i]
        self.biases[i] -= self.lr * grad_b[i]

# Training loop
def fit(self, X, y):
    for epoch in range(self.max_iter):
        perm = np.random.permutation(X.shape[0])
        X_s, y_s = X[perm], y[perm]

        for start in range(0, X.shape[0], self.batch_size):
            Xb = X_s[start : start + self.batch_size]
            yb = y_s[start : start + self.batch_size]
            self._forward(Xb)
            self._backward(yb)

        loss = np.mean((self._forward(X).flatten() - y) ** 2)
        self.loss_history.append(loss)

        if (epoch + 1) % 100 == 0:
            print(f"Epoch {epoch+1:4d}/{self.max_iter} MSE Loss: {loss:.4f}")

# Prediction
def predict(self, X):
    return self._forward(X).flatten()

```

Bayesian Model Code (R)

1.Code

```

# Load packages
library(rstanarm)
library(bayesplot)
library(ggplot2)

# Read data
data <- read.csv("dataset.csv")

# Keep only the variables needed for the Bayesian model
df <- data[, c("popularity", "danceability", "energy", "tempo", "loudness", "valence")]

# Remove missing values
df <- na.omit(df)

# Standardize predictors
predictor_cols <- c("danceability", "energy", "tempo", "loudness", "valence")
df[predictor_cols] <- scale(df[predictor_cols])

# Fit Bayesian regression model
bayes_model <- stan_glm(
  popularity ~ danceability + energy + tempo + loudness + valence,
  data = df,
  family = gaussian(),
  chains = 4,
  iter = 2000,
  seed = 123,
  refresh = 0
)

# Model summary
summary(bayes_model)

```